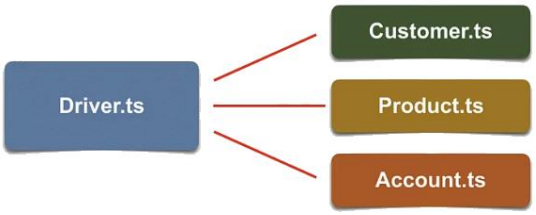
TypeScript Modules: Import and Export

In real-time projects, we often find ourselves dealing with a variety of classes and functions. In such scenarios, organizing the code is vital for maintainability and collaboration. In this lesson, we'll explore how TypeScript's module system facilitates this organization.

Understanding Modules:

Imagine you're working on a project with various components such as customer, product, and account. Currently, all your code resides in a single file, say `driver.ts`. This structure is not efficient for a scalable and maintainable application.



Here's where TypeScript's concept of modules comes into play.

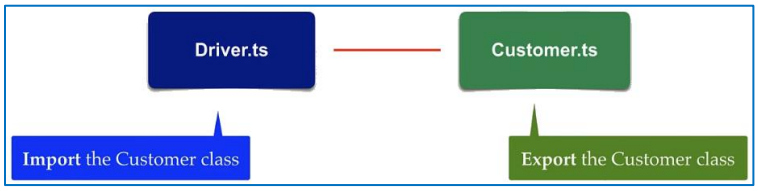
What Are Modules?

Modules in TypeScript allow you to break down your code into separate files. These files can then be included or imported into your application.

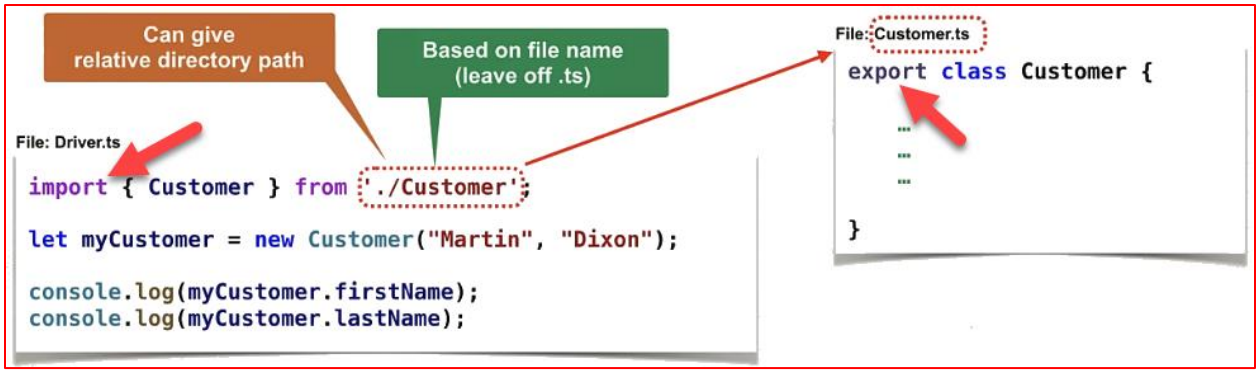
- **Exporting**: A module can export classes, functions, and variables.
- **Importing**: Another file can import these classes, functions, or variables from a given module.

An Example with `customer` and `driver` Files:

Let's walk through an example where we want to export the `Customer` class from the `customer.ts` file and import it into the `driver.ts` file.



- **Exporting the Class**: In the `customer.ts` file, we define the `Customer` class and export it by using the `export` keyword. This makes it available for other files to import.
- **Importing the Class**: In the `driver.ts` file, we import the `Customer` class using `import { Customer } from './customer';`. Note that you only provide the name of the file without the extension, and the `./` indicates that both files are in the same directory.

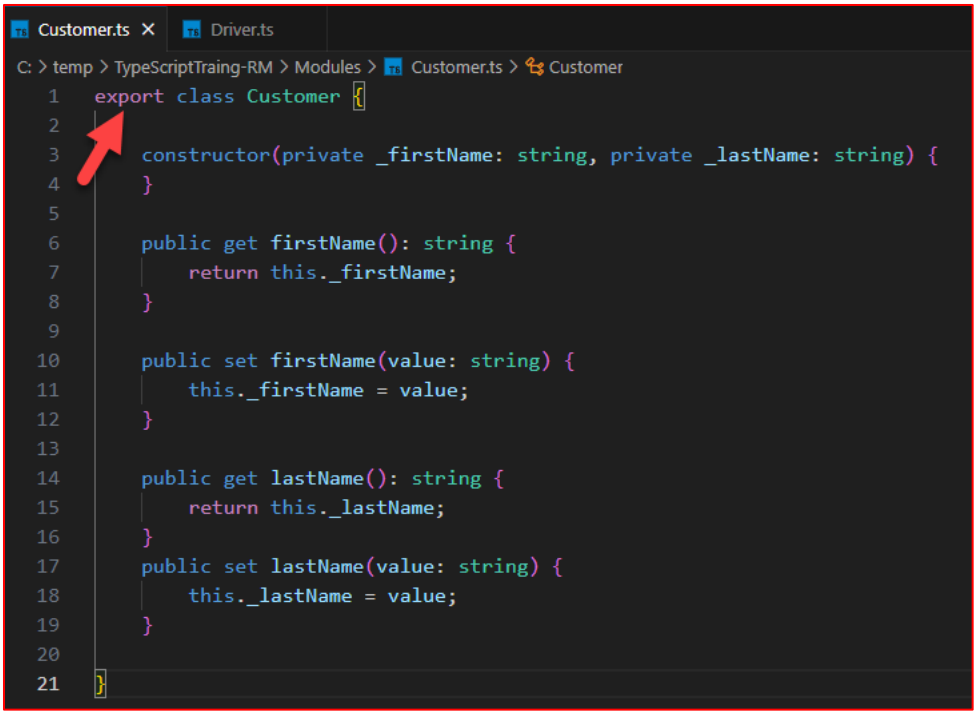


If the files are in different directories, you simply give a relative directory path. For instance, you can access subdirectories with `sub1/sub2/sub3` or move up directories with `../`.

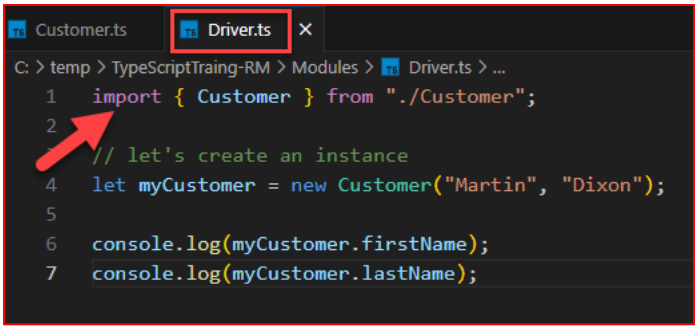
Putting It into Practice:

Let's delve into coding and see how this works.

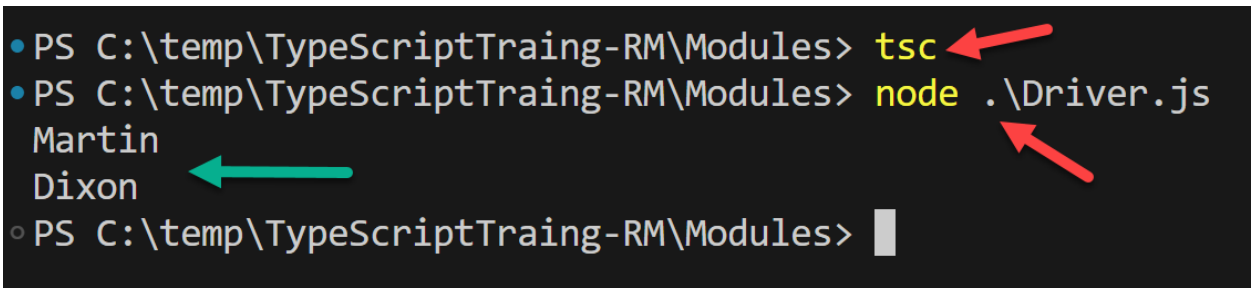
1. **Creating `driver.ts` and `customer.ts` Files**: We'll begin by creating these two files in the same directory.



2. **Exporting `Customer` Class**: Go to `customer.ts` and use the `export` keyword to make the class available for import.
3. **Creating Main Application in `driver.ts`**: Open `driver.ts`, and import the `Customer` class from `customer.ts`. If you encounter a compilation error saying "cannot find customer," it's likely that you missed exporting the class in `customer.ts`.



4. **Testing the Code**: Open a terminal window, move to the appropriate directory, compile, and run the code. You should see the desired output, demonstrating that the driver program can successfully import the `Customer` class and create an instance of it.



Benefits of Using Modules:

- **Enhanced Code Organization**: By dividing code into separate files, you facilitate collaboration and maintainability.
- **Scalability**: As your project grows, modules enable you to manage complexity more effectively.
- **Reusability**: Modules allow for easy reuse of classes, functions, or variables across different parts of your application.

Summary:

In this lesson, we've explored how TypeScript's module system can transform a messy, single-file codebase into a well-structured, scalable, and maintainable application. The ability to export and import classes, functions, and variables promotes a clean separation of concerns.

We encourage you to experiment with this in your projects. By mastering modules, you'll take a significant step towards writing professional and robust TypeScript code.